



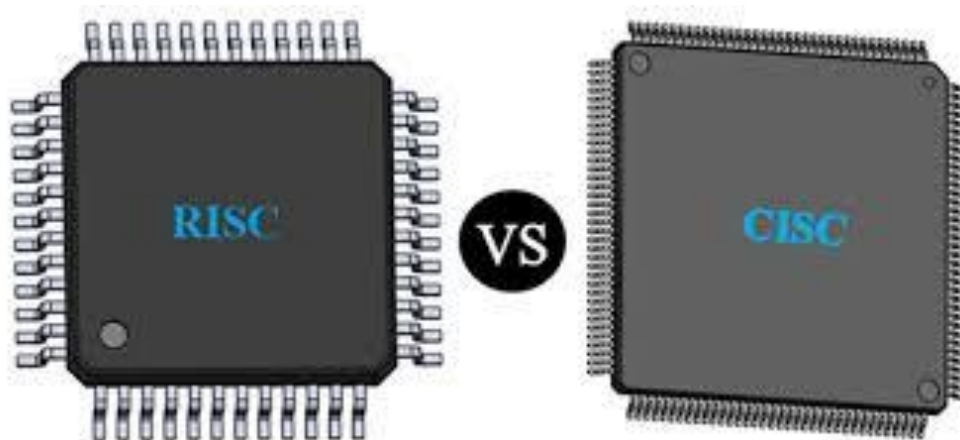
Actividad 05 - ISA y Endianness

Octavio Emmanuel López Ortiz

221933767

26/02/2026

Arquitectura de computadoras – D03



Introducción:

En el estudio de la arquitectura de computadoras, la Arquitectura del Conjunto de Instrucciones (ISA, Instruction Set Architecture) constituye el nivel fundamental que define la comunicación entre el hardware y el software. La ISA especifica el conjunto de instrucciones que el procesador puede ejecutar, los tipos de datos que maneja, los registros disponibles, los modos de direccionamiento y el formato de las instrucciones. En otras palabras, representa el contrato entre el programador y el procesador, permitiendo que los programas puedan ejecutarse en una determinada familia de sistemas.

¿Que es la ISA?

La arquitectura del conjunto de instrucciones (por sus siglas en ingles ISA) es la interfaz conceptual y abstracta entre el software y hardware de un procesador. Es la que define el conjunto de comandos, tipos de datos, registros y modos de memoria que la CPU entiende y ejecuta, permitiendo que el software funcione en distintos procesadores con la misma ISA.

Se podría decir que la ISA es el lenguaje básico que el software utiliza para comunicarse con el hardware. Esta incluye instrucciones para realizar operaciones aritméticas, lógicas y el movimiento de datos. Define la cantidad y tipo de registros accesibles, así como el modelo de acceso a la memoria. También especifica cómo el procesador maneja la entrada y salida de datos.

Los ejemplos principales incluyen x86 (computadoras/servidores), ARM (móviles/laptops modernas), RISC-V (código abierto), MIPS y PowerPC, clasificados mayormente en las familias RISC o CISC. La arquitectura CISC x86 es un diseño de procesadores (desarrollado mayormente por Intel y AMD) que utiliza un "Conjunto de Instrucciones Complejas" (Complex Instruction Set Computer), permitiendo ejecutar operaciones sofisticadas y múltiples pasos con una sola instrucción. Es la que mas se utiliza en computadoras personales y servidores, priorizando la potencia y la compatibilidad histórica sobre la eficiencia energética. A diferencia de RISC, una sola instrucción CISC puede cargar, operar y guardar datos en la memoria.

Aunque son CISC, los procesadores x86 modernos suelen convertir estas instrucciones complejas en microinstrucciones más simples (estilo RISC) internamente para mayor eficiencia. Reduce el tamaño del código ensamblador, facilita la creación de compiladores y mantiene la compatibilidad con software antiguo. x86 es el estándar de instrucciones, mientras que CISC es la filosofía de diseño compleja subyacente que permite a las CPU de Intel y AMD ejecutar tareas pesadas de manera eficiente.

Por otro lado la arquitectura ARM (Advanced RISC Machine) es un diseño de procesador basado en el concepto RISC (Reduced Instruction Set Computer) que utiliza instrucciones simples, eficientes y rápidas, optimizadas para el bajo consumo energético y alta eficiencia. Domina el mercado de dispositivos móviles, IoT y embebidos, destacando por su alta capacidad de integración y rendimiento por vatio, siendo fundamental en smartphones y, más recientemente, en ordenadores y servidores.

Utiliza un número limitado de instrucciones simples que, por lo general, se ejecutan en un solo ciclo de reloj. Siendo esto mas eficiente que el x86. Su diseño está optimizado para consumir menos energía, lo que lo convierte en el estándar para dispositivos que funcionan con baterías. Las instrucciones que procesan datos operan únicamente sobre registros, separando las operaciones de memoria de las operaciones de procesamiento. Ofrece alta compatibilidad, escalabilidad y seguridad para diversas aplicaciones, desde microcontroladores hasta supercomputadoras. Combina núcleos de alto rendimiento con núcleos eficientes en el mismo chip, optimizando el rendimiento y la duración de la batería según la tarea.

Diferencia entre RISC y CISC:

La principal diferencia radica en la complejidad y longitud de las instrucciones: RISC (Reduced Instruction Set Computer) utiliza instrucciones simples y uniformes que se ejecutan en un solo ciclo, priorizando la velocidad, mientras que CISC (Complex Instruction Set Computer) utiliza instrucciones complejas y variables que pueden tomar múltiples ciclos, priorizando la reducción del número de instrucciones por programa.

RISC se centra en la ejecución rápida de instrucciones básicas, a menudo completando una por ciclo de reloj. CISC busca realizar operaciones complejas con menos líneas de código, lo que puede requerir más ciclos. RISC tiene un diseño más simple y eficiente energéticamente, ideal para dispositivos móviles (ej. ARM). CISC tiene un diseño más complejo, utilizado comúnmente en computadoras de escritorio (ej. x86). RISC depende más de los registros y a menudo requiere más memoria para almacenar los programas. CISC suele requerir menos memoria. Los procesadores RISC están altamente segmentados, lo que facilita el procesamiento paralelo.

Endianness de mi computadora:

Para poder conocer si la computadora usa Little Endian o Big Endian, utilizaremos un pequeño código en C para poder detectar donde se guarda el byte más o menos significativo. Haciendo la prueba en mi computadora sale como resultado Little Endian, ya que es el más común actualmente.

```
main.c x
1  #include <stdio.h>
2  int main() {
3      unsigned int i = 1;
4      char *c = (char*)&i;
5      if (*c)
6          printf("Little endian\n");
7      else
8          printf("Big endian\n");
9      return 0;
10 }
11
```

"C:\Users\tavol\OneDrive\De: x + v

Little endian

Process returned 0 (0x0) execution time : 0.010 s
Press any key to continue.

Figura 1. Testeo del código

Se realizó un ejemplo práctico en el cuaderno para poder visualizar cómo se guardaba la información según si es Little endian o big endian, dicha práctica se puede observar en la siguiente imagen:

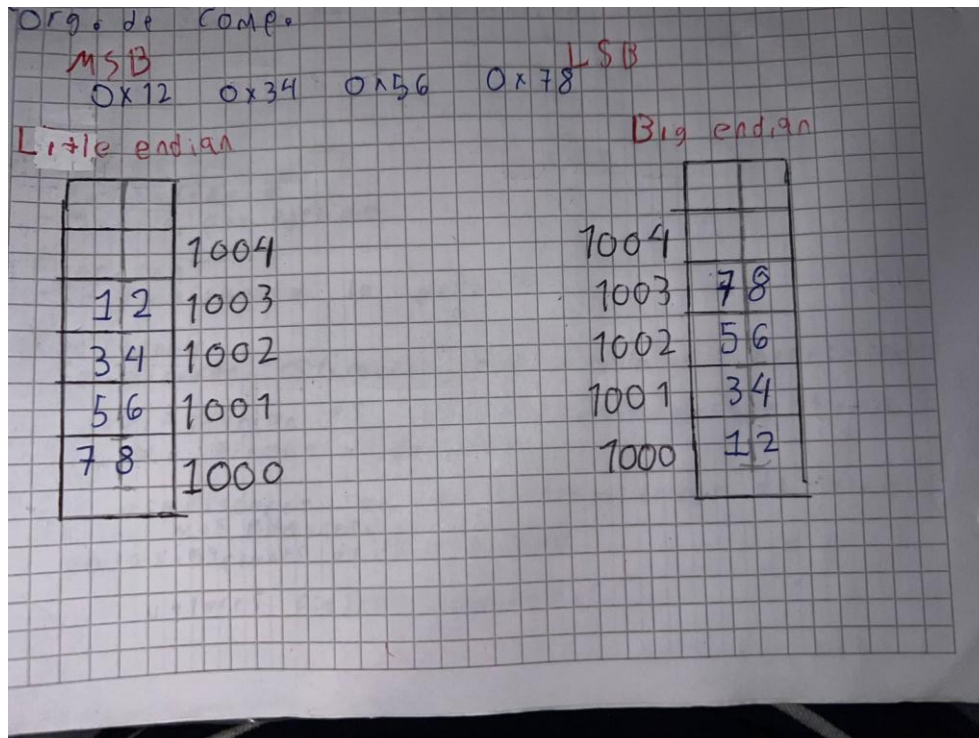


Figura 2. Fotografía de práctica

Bibliografía:

1. *Conceptos básicos de la arquitectura del conjunto de instrucciones* | *Lenovo España*. (s. f.). Recuperado 26 de febrero de 2026, de <https://www.lenovo.com/es/es/glossary/instruction-set-architecture/?orgRef=https%253A%252F%252Fwww.google.com%252F>
2. Arm Ltd. (s. f.). *What is Instruction Set Architecture (ISA)?* Arm | The Architecture For The Digital World. Recuperado 26 de febrero de 2026, de <https://www.arm.com/glossary/isa#:~:text=Una%20ISA%20define%20el%20comportamiento,y%20realizando%20las%20operaciones%20definidas>.
3. Shima. (2024, 14 marzo). *¿Que es ISA?* Medium. Recuperado 26 de febrero de 2026, de <https://medium.com/@shb8086/tutorial-series-isa-fa9648c5e9e5>
4. Anand. (s. f.). *How to check whether a system is big endian or little endian?* Stack Overflow. Recuperado 26 de febrero de 2026, de <https://stackoverflow.com/questions/4181951/how-to-check-whether-a-system-is-big-endian-or-little-endian>
5. *x86: ¿qué es?* | *Lenovo México*. (s. f.). Recuperado 26 de febrero de 2026, de <https://www.lenovo.com/mx/es/glosario/x86/?orgRef=https%253A%252F%252Fwww.google.com%252F>
6. 0xr4m! (2024, 8 noviembre). *CISC vs. RISC: The Differences Between x86 and ARM Architectures*. Medium. Recuperado 26 de febrero de 2026, de <https://medium.com/@samah.ikramfarez/cisc-vs-risc-the-differences-between-x86-and-arm-architectures-639a64ad7c76>
7. Delgado, A. (2020, 2 octubre). *¿Qué es ARM y para qué se usa?* Geeknetic. Recuperado 26 de febrero de 2026, de <https://www.geeknetic.es/ARM/que-es-y-para-que-sirve>
8. Team, S. R., & Team, S. R. (s. f.). *RISC vs. CISC – Decoding How RISC Architecture is Different from the CISC Architecture?* Stromasys. Recuperado 26 de febrero de 2026, de <https://www.stromasys.com/resources/decoding-risc-vs-cisc-architecture/#:~:text=Estos%20dos%20dise%C3%B1os%20difieren%20en,muchas%20acciones%20de%20bajo%20nivel>.
9. Cabe Atwell. (2024, 7 febrero). *RISC-V vs. ARM processors: A quick comparison*. Fierce Sensors. Recuperado 26 de febrero de 2026, de <https://www.fiercesensors.com/analog/risc-v-vs-arm-processors-quick-comparison#:~:text=%C2%BFQu%C3%A9%20es%20ARM?,a%20diferentes%20segmentos%20del%20mercado>.